ITA0605 - Machine Learning
List of Lab Exercises

4. Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

Aim:

To build an Artificial Neural Network (ANN) using the Backpropagation algorithm and test it using an appropriate dataset.

Dataset:

Use the **Breast Cancer Wisconsin Dataset**, which is available in **scikit-learn**, so no download is required.

Algorithm:

1. Import required libraries.
2. Load the dataset.
3. Split the data into training and testing sets.
4. Normalize the data.
5. Create an Artificial Neural Network.
6. Train the model using Backpropagation.
7. Test the model.
8. Display the accuracy.

Dataset 1:

Program Code:

```
from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split from

sklearn.preprocessing import StandardScaler from sklearn.neural_network

import MLPClassifier from sklearn.metrics import accuracy_score
```

```
data = load_breast_cancer()

X = data.data

y = data.target

X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2,

        random_state=42

)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train) X_test =

scaler.transform(X_test)

ann = MLPClassifier(

        hidden_layer_sizes=(10,),

        max_iter=1000, random_state=42

)

ann.fit(X_train, y_train) y_pred =

ann.predict(X_test)

accuracy = accuracy_score(y_test, y_pred) print("Accuracy :", round(accuracy

* 100, 2), "%")
```

Output:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

data = load_breast_cancer()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

ann = MLPClassifier(
    hidden_layer_sizes=(10,),
    max_iter=1000,
    random_state=42
)

ann.fit(X_train, y_train)

y_pred = ann.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy :", round(accuracy * 100, 2), "%")
```

```
Accuracy : 98.25 %
```

Dataset 2:

Program Code:

```
import pandas as pd
from sklearn.neural_network import MLPClassifier

data = [
    ['High',   'Good',    'Permanent', 'Yes', 'Yes'],
    ['High',   'Good',    'Permanent', 'No',  'Yes'],
    ['Medium', 'Good',    'Permanent', 'Yes', 'Yes'],
    ['Medium', 'Average', 'Permanent', 'No',  'Yes'],
    ['Low',    'Poor',    'Temporary', 'No',  'No'],
    ['Low',    'Average', 'Temporary', 'Yes', 'No'],
    ['High',   'Average', 'Permanent', 'Yes', 'Yes'],
    ['Medium', 'Good',    'Temporary', 'No',  'No'],
```

```
    ['High',   'Good',    'Permanent', 'Yes', 'Yes'],
    ['Low',    'Poor',    'Temporary', 'Yes', 'No']
]

columns = [
    'Income',
    'Credit Score',
    'Employment',
    'Property',
    'Loan Approved'
]

df = pd.DataFrame(data, columns=columns)

print("Original Dataset:")
print(df)


X = df[
    ['Income', 'Credit Score', 'Employment', 'Property']
]

y = df['Loan Approved'].map({
    'No': 0,
    'Yes': 1
})


X_encoded = pd.get_dummies(X)

print("\nEncoded Training Data:")
print(X_encoded)


model = MLPClassifier(
    hidden_layer_sizes=(10,),
    activation='relu',
    solver='lbfgs',
    max_iter=1000,
    random_state=42
)


model.fit(X_encoded, y)

print("\nANN Model Training Completed Successfully.")


new_sample = pd.DataFrame([
    ['Medium', 'Good', 'Permanent', 'Yes']
], columns=[
    'Income',
```

```
    'Credit Score',
    'Employment',
    'Property'
])

print("\nNew Sample:")
print(new_sample)


new_sample_encoded = pd.get_dummies(new_sample)

new_sample_encoded = new_sample_encoded.reindex(
    columns=X_encoded.columns,
    fill_value=0
)


prediction = model.predict(new_sample_encoded)

result = "Yes" if prediction[0] == 1 else "No"

print("\n-----------------------------------")
print("ANN Loan Approval Prediction")
print("-----------------------------------")

print("Predicted Result:", result)
print("-----------------------------------")
```

Output:

```
Original Dataset:
   Income Credit Score Employment Property Loan Approved
0    High         Good  Permanent      Yes            Yes
1    High         Good  Permanent       No            Yes
2  Medium         Good  Permanent      Yes            Yes
3  Medium      Average  Permanent       No            Yes
4     Low         Poor  Temporary       No             No
5     Low      Average  Temporary      Yes             No
6    High      Average  Permanent      Yes            Yes
7  Medium         Good  Temporary       No             No
8    High         Good  Permanent      Yes            Yes
9     Low         Poor  Temporary      Yes             No

Encoded Training Data:
   Income_High  Income_Low  Income_Medium  Credit Score_Average  \
0         True       False          False                 False
1         True       False          False                 False
2        False       False           True                 False
3        False       False           True                  True
4        False        True          False                 False
5        False        True          False                  True
6         True       False          False                  True
7        False       False           True                 False
8         True       False          False                 False
9        False        True          False                 False
```

```
   Credit Score_Good  Credit Score_Poor  Employment_Permanent  \
0               True              False                  True
1               True              False                  True
2               True              False                  True
3              False              False                  True
4              False               True                 False
5              False              False                 False
6              False              False                  True
7               True              False                 False
8               True              False                  True
9              False               True                 False

   Employment_Temporary  Property_No  Property_Yes
0                 False        False          True
1                 False         True         False
2                 False        False          True
3                 False         True         False
4                  True         True         False
5                  True        False          True
6                 False        False          True
7                  True         True         False
8                 False        False          True
9                  True        False          True
```

```
ANN Model Training Completed Successfully.

New Sample:
   Income Credit Score Employment Property
0  Medium         Good  Permanent      Yes

-----------------------------------
ANN Loan Approval Prediction
-----------------------------------
Predicted Result: Yes
-----------------------------------
```

Dataset 3:

Program Code:

```
import pandas as pd
from sklearn.neural_network import MLPClassifier

data = [
   ['Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'Positive'],
   ['Yes', 'Yes', 'No',  'Yes', 'Yes', 'Positive'],
   ['No',  'Yes', 'Yes', 'No',  'No',  'Negative'],
   ['Yes', 'Yes', 'Yes', 'No',  'Yes', 'Positive'],
   ['No',  'No',  'Yes', 'Yes', 'No',  'Negative'],
   ['Yes', 'Yes', 'No',  'No',  'Yes', 'Positive'],
   ['Yes', 'No',  'Yes', 'Yes', 'Yes', 'Positive'],
   ['No',  'No',  'No',  'No',  'No',  'Negative']
]

columns = [
```

```
    'Fever',
    'Cough',
    'Headache',
    'Body Pain',
    'Fatigue',
    'Disease'
]
df = pd.DataFrame(data, columns=columns)

print("Original Dataset:")
print(df)


X = df[
    ['Fever', 'Cough', 'Headache', 'Body Pain', 'Fatigue']
]

y = df['Disease'].map({
    'Negative': 0,
    'Positive': 1
})


X_encoded = X.replace({
    'No': 0,
    'Yes': 1
})

print("\nEncoded Training Data:")
print(X_encoded)


model = MLPClassifier(
    hidden_layer_sizes=(10,),
    activation='relu',
    solver='lbfgs',
    max_iter=1000,
    random_state=42
)


model.fit(X_encoded, y)

print("\nANN Model Training Completed Successfully.")


new_sample = pd.DataFrame([
    ['Yes', 'Yes', 'No', 'Yes', 'Yes']
], columns=[
    'Fever',
    'Cough',
    'Headache',
```

```
    'Body Pain',
    'Fatigue'
])

print("\nNew Sample:")
print(new_sample)


new_sample_encoded = new_sample.replace({
    'No': 0,
    'Yes': 1
})


prediction = model.predict(new_sample_encoded)


result = "Positive" if prediction[0] == 1 else "Negative"

print("\n-----------------------------------")
print("ANN Disease Diagnosis Prediction")
print("-----------------------------------")

print("Predicted Result:", result)
print("-----------------------------------")
```

Output:

```
Original Dataset:
  Fever Cough Headache Body Pain Fatigue   Disease
0   Yes   Yes      Yes       Yes     Yes  Positive
1   Yes   Yes       No       Yes     Yes  Positive
2    No   Yes      Yes        No      No  Negative
3   Yes   Yes      Yes        No     Yes  Positive
4    No    No      Yes       Yes      No  Negative
5   Yes   Yes       No        No     Yes  Positive
6   Yes    No      Yes       Yes     Yes  Positive
7    No    No       No        No      No  Negative

Encoded Training Data:
   Fever  Cough  Headache  Body Pain  Fatigue
0      1      1         1          1        1
1      1      1         0          1        1
2      0      1         1          0        0
3      1      1         1          0        1
4      0      0         1          1        0
5      1      1         0          0        1
6      1      0         1          1        1
7      0      0         0          0        0

ANN Model Training Completed Successfully.
```

Dataset 4:

Program Code:

```
import pandas as pd
from sklearn.neural_network import MLPClassifier

data = [
    ['High',   'Excellent', 'Yes', 'Yes', 'Promoted'],
    ['High',   'Good',      'Yes', 'Yes', 'Promoted'],
    ['Medium', 'Good',      'Yes', 'Yes', 'Promoted'],
    ['Medium', 'Average',   'No',  'Yes', 'Not Promoted'],
    ['Low',    'Poor',      'No',  'No',  'Not Promoted'],
    ['High',   'Excellent', 'Yes', 'No',  'Promoted'],
    ['Medium', 'Good',      'No',  'Yes', 'Promoted'],
    ['Low',    'Average',   'No',  'No',  'Not Promoted'],
    ['High',   'Good',      'Yes', 'Yes', 'Promoted'],
    ['Low',    'Poor',      'No',  'Yes', 'Not Promoted']
]

columns = [
    'Experience',
    'Performance',
    'Leadership',
    'Training',
    'Promotion'
]

df = pd.DataFrame(data, columns=columns)

print("Original Dataset:")
print(df)


X = df[
    ['Experience', 'Performance', 'Leadership', 'Training']
]

y = df['Promotion'].map({
    'Not Promoted': 0,
    'Promoted': 1
})

X_encoded = pd.get_dummies(X)

print("\nEncoded Training Data:")
print(X_encoded)


model = MLPClassifier(
    hidden_layer_sizes=(10,),
    activation='relu',
```

```
    solver='lbfgs',
    max_iter=1000,
    random_state=42
)


model.fit(X_encoded, y)

print("\nANN Model Training Completed Successfully.")

new_sample = pd.DataFrame([
    ['Medium', 'Good', 'Yes', 'Yes']
], columns=[
    'Experience',
    'Performance',
    'Leadership',
    'Training'
])

print("\nNew Sample:")
print(new_sample)

new_sample_encoded = pd.get_dummies(new_sample)

new_sample_encoded = new_sample_encoded.reindex(
    columns=X_encoded.columns,
    fill_value=0
)

prediction = model.predict(new_sample_encoded)


result = "Promoted" if prediction[0] == 1 else "Not Promoted"


print("\n-----------------------------------")
print("ANN Employee Promotion Prediction")
print("-----------------------------------")

print("Predicted Result:", result)
print("-----------------------------------")
```

Output:

```
Original Dataset:
  Experience Performance Leadership Training     Promotion
0       High   Excellent        Yes      Yes      Promoted
1       High        Good        Yes      Yes      Promoted
2     Medium        Good        Yes      Yes      Promoted
3     Medium     Average         No      Yes  Not Promoted
4        Low        Poor         No       No  Not Promoted
5       High   Excellent        Yes       No      Promoted
6     Medium        Good         No      Yes      Promoted
7        Low     Average         No       No  Not Promoted
8       High        Good        Yes      Yes      Promoted
9        Low        Poor         No      Yes  Not Promoted

Encoded Training Data:
   Experience_High  Experience_Low  Experience_Medium  Performance_Average  \
0             True           False              False                False
1             True           False              False                False
2            False           False               True                False
3            False           False               True                 True
4            False            True              False                False
5             True           False              False                False
6            False           False               True                False
7            False            True              False                 True
8             True           False              False                False
9            False            True              False                False
```

```
   Performance_Excellent  Performance_Good  Performance_Poor  Leadership_No  \
0                   True             False             False          False
1                  False              True             False          False
2                  False              True             False          False
3                  False             False             False           True
4                  False             False              True           True
5                   True             False             False          False
6                  False              True             False           True
7                  False             False             False           True
8                  False              True             False          False
9                  False             False              True           True

   Leadership_Yes  Training_No  Training_Yes
0            True        False          True
1            True        False          True
2            True        False          True
3           False        False          True
4           False         True         False
5            True         True         False
6           False        False          True
7           False         True         False
8            True        False          True
9           False        False          True
```

```
ANN Model Training Completed Successfully.

New Sample:
  Experience Performance Leadership Training
0     Medium        Good        Yes      Yes

-----------------------------------
ANN Employee Promotion Prediction
-----------------------------------
Predicted Result: Promoted
-----------------------------------
```

Dataset 5:

Program Code:

```
import pandas as pd
from sklearn.neural_network import MLPClassifier

data = [
    ['Yes', 'Yes', 'Yes', 'No',  'Yes', 'Spam'],
    ['Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'Spam'],
    ['No',  'No',  'No',  'No',  'No',  'Not Spam'],
    ['Yes', 'No',  'Yes', 'No',  'Yes', 'Spam'],
    ['No',  'Yes', 'No',  'Yes', 'No',  'Not Spam'],
    ['Yes', 'Yes', 'Yes', 'No',  'No',  'Spam'],
    ['Yes', 'No',  'Yes', 'Yes', 'Yes', 'Spam'],
    ['No',  'No',  'Yes', 'No',  'No',  'Not Spam'],
    ['Yes', 'Yes', 'No',  'No',  'Yes', 'Spam'],
    ['No',  'No',  'No',  'Yes', 'No',  'Not Spam']
]

columns = [
    'Contains Link',
    'Offer Words',
    'Unknown Sender',
    'Attachment',
    'Many Recipients',
    'Spam'
]

df = pd.DataFrame(data, columns=columns)

print("Original Dataset:")
print(df)


X = df[
    ['Contains Link',
     'Offer Words',
     'Unknown Sender',
     'Attachment',
```

```
    'Many Recipients']
]

y = df['Spam'].map({
    'Not Spam': 0,
    'Spam': 1
})


X_encoded = X.replace({
    'No': 0,
    'Yes': 1
})

print("\nEncoded Training Data:")
print(X_encoded)


model = MLPClassifier(
    hidden_layer_sizes=(10,),
    activation='relu',
    solver='lbfgs',
    max_iter=1000,
    random_state=42
)

model.fit(X_encoded, y)

print("\nANN Model Training Completed Successfully.")


new_sample = pd.DataFrame([
    ['Yes', 'Yes', 'Yes', 'No', 'Yes']
], columns=[
    'Contains Link',
    'Offer Words',
    'Unknown Sender',
    'Attachment',
    'Many Recipients'
])

print("\nNew Sample:")
print(new_sample)


new_sample_encoded = new_sample.replace({
    'No': 0,
    'Yes': 1
})

prediction = model.predict(new_sample_encoded)
```

```
result = "Spam" if prediction[0] == 1 else "Not Spam"

print("\n-----------------------------------")
print("ANN Email Spam Prediction")
print("-----------------------------------")

print("Predicted Result:", result)
print("-----------------------------------")
```

Output:

```
Original Dataset:
  Contains Link Offer Words Unknown Sender Attachment Many Recipients  \
0           Yes         Yes             Yes         No              Yes
1           Yes         Yes             Yes        Yes              Yes
2            No          No              No         No               No
3           Yes          No             Yes         No              Yes
4            No         Yes              No        Yes               No
5           Yes         Yes             Yes         No               No
6           Yes          No             Yes        Yes              Yes
7            No          No             Yes         No               No
8           Yes         Yes              No         No              Yes
9            No          No              No        Yes               No

       Spam
0      Spam
1      Spam
2  Not Spam
3      Spam
4  Not Spam
5      Spam
6      Spam
7  Not Spam
8      Spam
9  Not Spam
```

```
Encoded Training Data:
   Contains Link  Offer Words  Unknown Sender  Attachment  Many Recipients
0              1            1               1           0                1
1              1            1               1           1                1
2              0            0               0           0                0
3              1            0               1           0                1
4              0            1               0           1                0
5              1            1               1           0                0
6              1            0               1           1                1
7              0            0               1           0                0
8              1            1               0           0                1
9              0            0               0           1                0

ANN Model Training Completed Successfully.

New Sample:
  Contains Link Offer Words Unknown Sender Attachment Many Recipients
0           Yes         Yes             Yes         No              Yes

-----------------------------------
ANN Email Spam Prediction
-----------------------------------
Predicted Result: Spam
-----------------------------------
```

Result:

The Artificial Neural Network was successfully implemented using the **Backpropagation algorithm**. The model was trained and tested on the Breast Cancer dataset and achieved high classification accuracy.